

CSC 212: Data Structures and Abstractions

08: Queues and Deques

Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

Fall 2026



Motivation

- An inference server runs a model on one GPU
 - ✓ requests arrive from many users, faster than the GPU can answer
 - ✓ **fairness**: whoever asked first is served first
- The waiting requests form a line:
 - ✓ new requests join at the **back**
 - ✓ the GPU takes the next request from the **front**
- First idea: a `std::vector`

```
std::vector<Request> waiting;
waiting.push_back(r);           // a request arrives
Request next = waiting.front(); // the GPU is free
waiting.erase(waiting.begin()); // remove it from the line
```

2

Q8-1

- $n = 1,000,000$ requests are waiting in the vector, and the GPU serves all of them. About how many element moves do the `erase(waiting.begin())` calls cost in total?

- A. About 10^6
- B. About 2×10^6
- C. About 5×10^{11}
- D. About 10^{18}
- E. None of the above

3

Queues

Queues

- A **queue** is a container of elements that are inserted and removed according to the **first-in first-out (FIFO)** principle
 - ✓ elements can be inserted at any time
 - ✓ only the element that has been in the queue the **longest** can be removed at any time
 - ✓ elements enter the queue at the **rear** and are removed from the **front**
- The fundamental operations are **enqueue** and **dequeue**
- Everyday queues
 - ✓ requests to a web server or a model
 - ✓ jobs sent to a printer
 - ✓ processes waiting for the CPU
 - ✓ packets waiting in a router
 - ✓ graph algorithms



5

The queue ADT

- An **abstract data type (ADT)** specifies what each operation does, not how (*the contract*)
 - ✓ the data structure decides how and at which cost
- The queue ADT supports:

operation	what it does
enqueue(e)	insert element e at the rear of the queue
dequeue()	remove the element at the front of the queue; an error occurs if the queue is empty
front()	return, but do not remove, a reference to the front element; an error occurs if the queue is empty
size()	return the number of elements in the queue
empty()	return true if the queue is empty, and false otherwise

The STL names: push is enqueue, pop is dequeue, and back() also reads the rear element

6

Q8-2

- Starting from an empty queue, run:
 - ✓ enqueue(5) enqueue(3) dequeue() enqueue(2)
 - ✓ enqueue(8) dequeue() enqueue(9) enqueue(1)
 - ✓ what does the queue hold, from front to rear?
- A. 5, 9, 1
B. 8, 9, 1
C. 2, 8, 9, 1
D. 1, 9, 8
E. None of the above

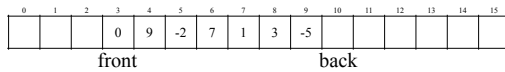
7

Implementation

- Array-based (standard)
 - ✓ enqueue at the end: $\Theta(1)$ cost (amortized if using a dynamic array)
 - ✓ dequeue at front: $\Theta(n)$ cost
- Array-based (alternative)
 - ✓ enqueue at front: $\Theta(n)$ cost
 - ✓ dequeue at end: $\Theta(1)$ cost
- Circular array
 - ✓ enqueue at end: $\Theta(1)$ cost (amortized if using a dynamic array)
 - ✓ dequeue at front: $\Theta(1)$ cost
 - ✓ **efficient approach**, as it eliminates the need for shifting elements
 - (*) requires handling **wrap-around** at array boundaries

8

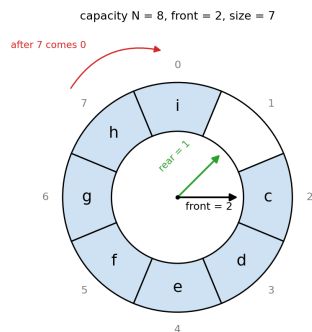
Circular array



<https://www.cs.usfca.edu/~galles/visualization/QueueArray.html>

9

Circular array



10

Q8-3

- A circular array has capacity = 8, front = 6, and size = 4. At which index does the next **enqueue** write?

- A. 10
- B. 2
- C. 3
- D. 6
- E. None of the above

11

The Queue class

```
class Queue {
private:
    int* m_data;           // circular array of m_capacity ints
    size_t m_front;        // index of the front element
    size_t m_size;         // elements stored
    size_t m_capacity;     // slots allocated

    void grow();

public:
    Queue() : m_data(new int[1]), m_front(0), m_size(0), m_capacity(1) {}
    ~Queue() { delete[] m_data; }

    Queue(const Queue&) = delete;           // copying
    Queue& operator=(const Queue&) = delete;

    void enqueue(int e);
    void dequeue();
    const int& front() const;
    size_t size() const { return m_size; }
    bool empty() const { return m_size == 0; }
};
```

= delete forbids copying, so double free cannot happen

12

The Queue class

```
void Queue::enqueue(int e) {
    if (m_size == m_capacity)
        grow();
    size_t rear = (m_front + m_size) % m_capacity; // first free slot
    m_data[rear] = e;
    ++m_size;
}

void Queue::dequeue() {
    if (empty())
        throw std::underflow_error("dequeue of empty queue");
    m_front = (m_front + 1) % m_capacity; // move the index, not the data
    --m_size;
}

const int& Queue::front() const {
    if (empty())
        throw std::underflow_error("front of empty queue");
    return m_data[m_front];
}

void Queue::grow() {
    size_t new_cap = 2 * m_capacity;
    int* bigger = new int[new_cap];
    for (size_t i = 0; i < m_size; ++i)
        bigger[i] = m_data[(m_front + i) % m_capacity]; // unroll, in queue order
    delete[] m_data;
    m_data = bigger;
    m_front = 0;
    m_capacity = new_cap;
}
```

13

Q8-4

• A full circular array of capacity 4 has front = 3 and holds the queue w, x, y, z. A buggy `grow` copies index `i` of the old array to index `i` of a new array of capacity 8, and keeps `f` = 3. In what order do the next four `dequeue` calls see the slots?

- A. w, x, y, z
- B. w, then three empty slots
- C. z, y, x, w
- D. x, y, z, w
- E. None of the above

14

Cost of each operation

operation	circular array, fixed	circular array, growing
enqueue(e)	$\Theta(1)$, error when full	$\Theta(1)$ amortized, $\Theta(n)$ worst case
dequeue()	$\Theta(1)$	$\Theta(1)$
front()	$\Theta(1)$	$\Theta(1)$
size(), empty()	$\Theta(1)$	$\Theta(1)$
space	$\Theta(n)$	$\Theta(n)$

15

std::queue

- `std::queue` is a **container adapter**
 - ✓ the underlying container is `std::deque` by default

```
#include <iostream>
#include <queue>
#include <string>

int main() {
    std::queue<std::string> q;
    q.push("Alice");
    q.push("Bob");
    q.push("Charlie");
    // Alice Charlie 3
    std::cout << q.front() << ' ' << q.back() << ' ' << q.size() << '\n';
    q.pop();
    std::cout << q.front() << '\n'; // Bob
}
```

16

Q8-5

What does this print?

- A. "300 -1 -1 | 200 200"
- B. "300 -1 100 | 300 200"
- C. "100 200 300 | -1 -1"
- D. "-1 -1 300 | 200 200"
- E. None of the above

```
std::queue<int> q1, q2;

q1.push(-1);
q2.push(-1);
q1.push(100);
q2.push(q1.front());
q1.pop();
q1.push(200);
q1.push(300);
q2.push(q1.front());
q1.pop();
q2.push(q1.front());
q1.pop();

q1.push(q2.front());
q2.pop();
q1.push(q2.front());
q2.pop();

while (!q1.empty()) {
    std::cout << q1.front() << ' ';
    q1.pop();
}

std::cout << " | ";
while (!q2.empty()) {
    std::cout << q2.back() << ' ';
    q2.pop();
}
```

17

Dequeues

Dequeues

- A double-ended queue, or **deque** (pronounced “deck”), supports insertion and deletion at both the front and the rear
 - it can act as a stack, as a queue, or as both at once



operation	STL name	what it does
insertFront(e)	push_front	insert e at the beginning
insertBack(e)	push_back	insert e at the end
eraseFront()	pop_front	remove the first element; error if empty
eraseBack()	pop_back	remove the last element; error if empty
front(), back()	same	return the first or the last element; error if empty
size(), empty()	same	as for stacks and queues

19

Example application

- A model can only read a limited amount of text, its **context window**
- A simple chat keeps the most recent k messages
 - a new message arrives: **push_back**
 - over the limit: drop the oldest with **pop_front**
 - regenerate the last answer: **pop_back**, then generate again
- Python has it built in:

```
from collections import deque

window = deque(maxlen=3)
for m in ["hi", "what is a stack?", "and a queue?", "thanks!"]:
    window.append(m)
print(list(window))  # ['what is a stack?', 'and a queue?', 'thanks!']
```

dropping messages from the front of an array in $\Theta(1)$ time

20

A deque on a circular array

- Same circular array as the queue. `push_back` is enqueue, `pop_front` is dequeue
- `push_front` steps the front index back by one, wrapping from 0 to capacity - 1

```
void Deque::push_front(int e) {  
    if (m_size == m_capacity)  
        grow();  
    m_front = (m_front + m_capacity - 1) % m_capacity; // step back, wrapping  
    m_data[m_front] = e;  
    ++m_size;  
}
```

21

Q8-6

- `m_front` and `m_capacity` are `size_t`, with `m_front = 0` and `m_capacity = 10`. What does `(m_front-1) % m_capacity` evaluate to?

- A. 9
- B. -1
- C. 5
- D. Undefined behavior
- E. None of the above

22

Exercises

- Write a function that modifies a queue of elements by replacing every element with two copies of itself
 - ✓ e.g., [a, b, c] becomes [a, a, b, b, c, c]
- Write a function that accepts a queue and appends the queue's contents to itself in reverse order
 - ✓ hint: can use a separate stack
 - ✓ e.g.: [a, b, c] becomes [a, b, c, c, b, a]
- Sliding window maximum: given an array and a window size k , output the maximum of every window of k consecutive elements
 - ✓ array: 1 3 -1 -3 5 3 6 7 $k = 3$
 - ✓ output: 3 3 5 5 6 7
 - ✓ brute force costs $\Theta(nk)$
 - ✓ reach $\Theta(n)$ with a deque of indices

23